NMIMS — Narsee Monjee Institute of Management Studies
School of Technology Management & Engineering, Indore

MINI-PROJECT REPORT

SignBridge

Real-Time Sign Language Recognition Assistant

Powered by YOLOv5

Course: Image and Video Processing

Student Name(s)	Aryan Soni, Manan Mali, Rounak Joshi
Roll Number(s)	F-135, F-134, F-151
Programme / Semester / Division	B.tech CE, Sec A, Batch 1
Faculty Name	Dr. Raj Gaurav Mishra
Institute / Department	SVKM's NMIMS, Indore — STME
Academic Year	2026–2027

Abstract

Communication barriers between deaf or hard-of-hearing individuals and people who do not know sign language remain a daily challenge in classrooms, offices and public services. This project presents SignBridge, a desktop application that recognises sign-language gestures from a live webcam feed and converts them into on-screen text and spoken audio in real time. A custom YOLOv5s-cls classifier was fine-tuned through transfer learning on the CollectedImages dataset of 1,860 images covering 31 conversational signs. The pipeline combines video frame capture, frame sampling, mirror correction, resizing, colour-space conversion and ImageNet normalisation, followed by CNN-based feature extraction and classification. A temporal state-machine stabiliser (5-frame window, 3-frame agreement, 40% confidence) removes jitter and duplicate triggers before a word is accepted, appended to a transcript and dictated through native text-to-speech. The model achieved 100% validation Top-1 and Top-5 accuracy with about 5.7 ms CPU and 0.7 ms GPU latency per image, enabling smooth real-time use. The application is supported by a PySide6 interface, a searchable sign library and a suite of 113 passing automated tests.

1. Introduction

1.1 Background of the Problem

Sign language is the primary means of communication for millions of deaf and hard-of-hearing people worldwide. Most hearing people, however, are not trained in it, so everyday interactions such as asking for help, answering in a classroom or greeting a colleague often depend on a human interpreter, who is not always available. Technology that can interpret signs automatically can reduce this dependence and make communication more inclusive.

1.2 Relevance of the Problem

- Inclusion: assistive tools help deaf and hard-of-hearing users take part in education, healthcare, workplaces and public services without a human interpreter.
- Availability: interpreters are costly and scarce, while a webcam-based tool needs only an ordinary laptop.
- Real-time need: conversation is fast, so a useful system must recognise and speak signs with minimal delay.

1.3 Role of Image and Video Processing

A sign is a visual event, so the problem is naturally one of image and video processing. A webcam provides a video stream that is broken into frames; each frame is pre-processed (colour conversion, resizing, normalisation) and analysed by a convolutional neural network that extracts hand-shape and pose features and classifies them. Because a single frame can be noisy, temporal processing across consecutive frames is used to decide when a gesture is genuinely present. Image and video processing therefore forms every stage of the system, from capture to decision.

1.4 Motivation

The motivation for this project was to build a practical, end-to-end assistive application rather than only a trained model. We wanted to apply the concepts learned in the Image and Video Processing course, namely frame handling, pre-processing, feature extraction, classification and temporal filtering, to a socially useful problem, and to test whether a lightweight YOLOv5 classifier could run in real time on an ordinary CPU.

2. Problem Statement and Objectives

2.1 Problem Statement

Hearing individuals who do not know sign language find it difficult to understand deaf and hard-of-hearing people, and human interpreters are not always available. The real-world problem addressed in this project is to automatically recognise a set of common conversational sign gestures from a live video stream and convey their meaning to a listener as text and speech, in real time and on commodity hardware.

2.2 Objectives

The objective of this project is to recognise conversational sign gestures from a live webcam video using image and video processing and a YOLOv5 deep-learning classifier. The specific objectives are:

1. To build a labelled dataset of 31 conversational signs and prepare stratified train, validation and test splits.
2. To fine-tune a YOLOv5s-cls model by transfer learning and reach high classification accuracy with CPU latency low enough for real-time use.
3. To design a temporal stabilisation mechanism that suppresses flicker and accidental duplicate detections in the video stream.
4. To convert recognised signs into a running transcript and spoken output using native text-to-speech.
5. To deliver a usable desktop interface with live camera view, confidence display, transcript tools and a searchable sign library.

3. Scope of the Project

3.1 What the Project Covers

- Recognition of 31 conversational sign gestures (for example again, book, please, stop, where, and i need help) from a live webcam.
- Dataset preparation, training, evaluation and confusion-matrix generation through reproducible scripts.
- Real-time prediction with temporal stabilisation, a conversation transcript with copy and clear options, and text-to-speech dictation (Play All, Play Last, click-to-dictate).
- A modern dark-themed PySide6 desktop application for Windows and macOS.

3.2 What the Project Does Not Cover

- Continuous sentence-level sign language translation or grammar generation; each recognised sign is treated as an individual word or phrase.
- Vocabulary beyond the 31 trained classes, finger-spelling of alphabets or regional sign languages not present in the dataset.
- Hand localisation or tracking with bounding boxes; the model classifies the whole frame.
- Mobile or web deployment, and speech-to-sign conversion in the opposite direction.

3.3 Type of Data and Application Domain

The system uses still colour images (1280×720) for training and a live RGB webcam video stream at about 30 FPS for inference. The application domain is assistive technology and inclusive education, including classrooms, help desks and reception counters, where a quick two-way exchange with a deaf or hard-of-hearing person is needed.

4. Dataset / Input Description

4.1 Source and Composition

The dataset, named CollectedImages, was collected by the project team by capturing images of each sign gesture. It contains 1,860 images distributed uniformly across 31 classes, with 60 images per class. All images share a uniform resolution of 1280×720 pixels. The 31 classes are: again, agree, answer, attendance, book, break, careful, congratulations, good morning, happy birthday, home, how are you, hungry, i need help, keepsmile, meet, mistake, please, practice, pressure, sick, stop, sun, team, thirsty, this, together, understand, wait, where and write.

Table 1: Dataset summary

Attribute	Description
Dataset name	CollectedImages (self-collected)
Total images	1,860
Number of classes	31 conversational signs
Images per class	60 (balanced)
Image resolution	1280 × 720 pixels (uniform)
Image format	[JPG / PNG — confirm from your dataset folder]
Model input size	224 × 224 RGB (after resizing)
Train / Val / Test split	70% / 15% / 15% — 1,302 / 279 / 279 images (stratified)
Live input (video)	Webcam stream at about 30 FPS, sampled every 200 ms

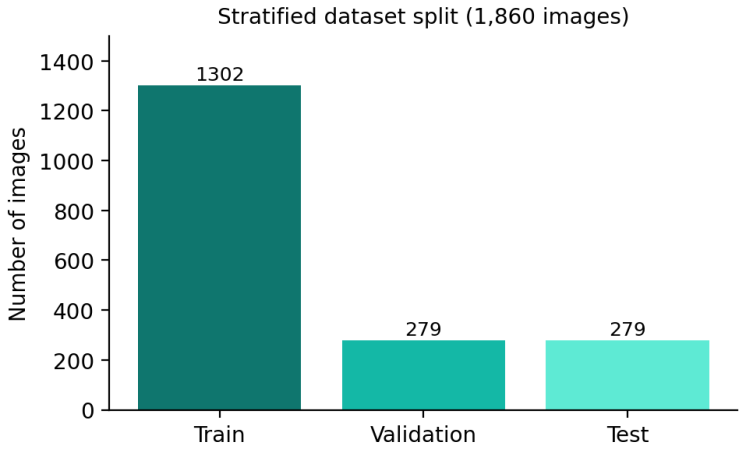


Figure 1: Stratified train / validation / test split

4.2 Sample Input Image





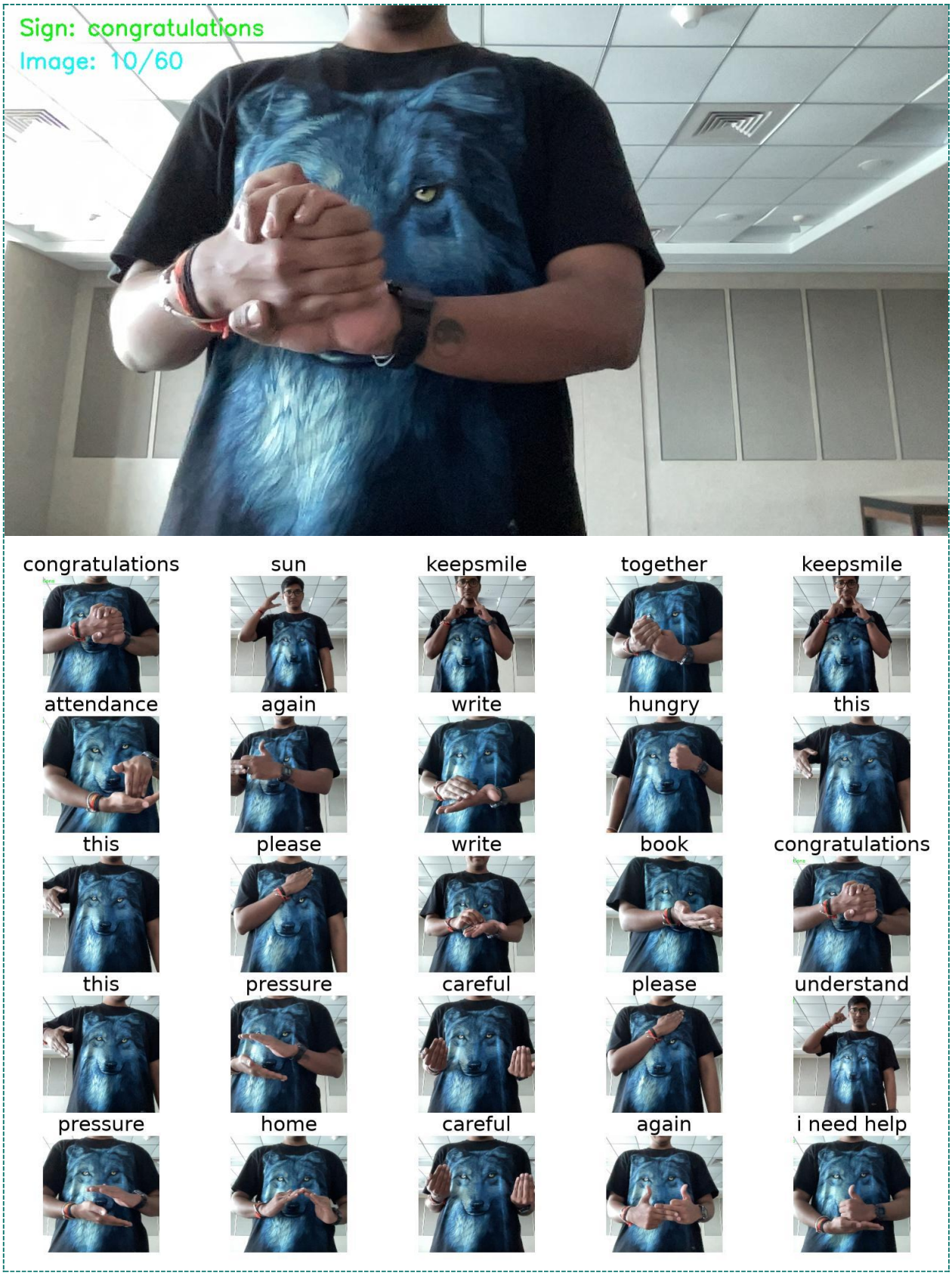


Figure 2: Sample input images from the CollectedImages dataset

5. Methodology /Proposed Approach

The system processes video as a continuous pipeline, shown in Figure 3. Each stage is described below.

1. Video capture: a thread-safe OpenCV module reads the webcam continuously at about 30 FPS so that the user interface never blocks.
2. Frame sampling and mirror correction: inference runs once every 200 ms to keep the CPU load low; the frame is flipped horizontally so that the on-screen view behaves like a mirror.
3. Pre-processing: the frame is converted from BGR to RGB, resized to 224×224, converted to a tensor and normalised with ImageNet mean and standard deviation.
4. Classification: the YOLOv5s-cls network produces a score for each of the 31 classes; the top label and its confidence are taken as the frame-level prediction.
5. Temporal stabilisation: frame-level predictions pass through a state machine that accepts a sign only when it is consistent across several frames.
6. Output: the confirmed word is added to the transcript, spoken aloud by the text-to-speech engine, and shown on the interface.

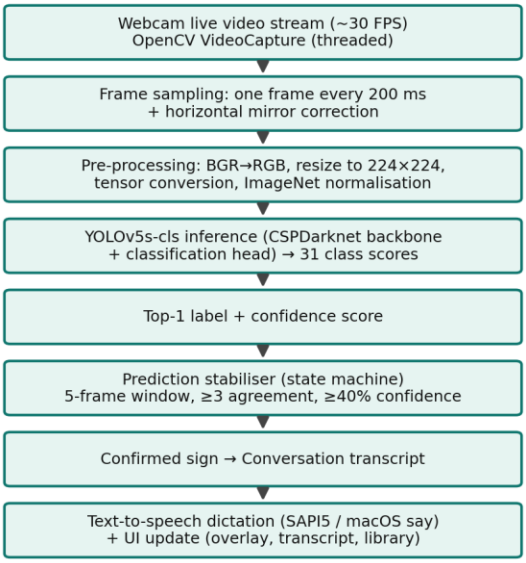


Figure 3: Block diagram of the SignBridge processing pipeline

Algorithm (pseudocode)

```
initialise model, stabiliser, transcript, TTS engine
open webcam
while camera is running:
    frame = read_frame();  frame = flip_horizontal(frame)
    every 200 ms:
        x = normalise(to_tensor(resize(bgr_to_rgb(frame), 224, 224)))
        scores = model(x);  label, conf = argmax(scores), max(scores)
        state = stabiliser.update(label, conf)
        if state == CONFIRMED:
            transcript.append(label);  tts.speak(label)
    update UI (overlay, confidence bar, transcript)
```

6. Techniques / Algorithms Used

6.1 Image Processing Techniques

Table 2: Image processing techniques used

Technique	Purpose in SignBridge
Colour-space conversion	OpenCV frames arrive in BGR; they are converted to RGB as expected by the network.
Resizing / rescaling	Frames and training images (1280×720) are resized to 224×224 for the network input.
Normalisation	Pixel values are scaled and standardised with ImageNet mean and standard deviation to match pre-trained weights.
Geometric transform (flip)	Horizontal mirror correction so that the live view matches the user's movement.
Feature extraction (CNN)	The CSPDarknet backbone learns hierarchical features such as edges, hand contours and shapes automatically.
Classification	The classification head maps the features to 31 class probabilities.
Transfer learning	Weights pre-trained on ImageNet (yolov5s-cls.pt) are fine-tuned on the sign dataset.

6.2 Video Processing Techniques

- **Frame extraction and sampling:** frames are read at about 30 FPS and one frame every 200 ms is analysed.
- **Temporal smoothing (prediction stabilisation):** a four-state machine (WAITING → DETECTING → CONFIRMED → COOLDOWN) confirms a sign only when at least 3 of the last 5 frames agree and the average confidence is at least 40%. A cooldown of 4 frames after acceptance prevents accidental duplicates while still allowing a sign to be repeated deliberately. Lowering the hands or switching gesture resets the state immediately.

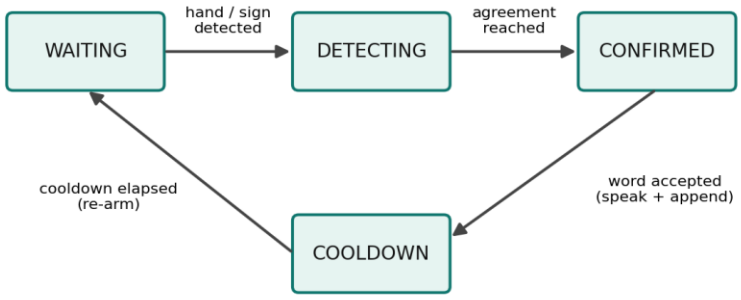


Figure 4: Prediction stabilisation state machine

Classical operations such as background subtraction, optical flow or morphological filtering were not used, because the deep network learns the required hand features directly and signs in this dataset are static gestures.

6.3 YOLOv5s-cls Model

YOLOv5s-cls is the classification variant of Ultralytics YOLOv5. It uses a CSPDarknet backbone with a classification head, has 4,206,399 parameters across 67 layers and requires 10.4 GFLOPs. Training started from ImageNet pre-trained weights, which allows a small dataset to reach high accuracy quickly.

7. Implementation

7.1 Programming Language, Tools and Libraries

Table 3: Implementation environment

Item	Details
Language	Python 3.13
Deep learning	PyTorch, Ultralytics YOLOv5 (yolov5s-cls)
Image / video	OpenCV (camera capture, colour conversion, resizing), NumPy
User interface	PySide6 (Qt) with a dark theme and 65% / 35% split layout
Speech	Windows SAPI5 (win32com) and macOS native say command
Evaluation	scikit-learn / Matplotlib for metrics and confusion matrix [confirm]
Testing	pytest — 113 automated tests
Hardware	[Laptop model, CPU, RAM, GPU if any, webcam]

7.2 Important Implementation Steps

1. Dataset preparation: prepare_yolov5_dataset.py splits CollectedImages into stratified train, validation and test folders (70/15/15) and writes dataset_config.yaml.
2. Training: train_yolov5.py fine-tunes yolov5s-cls.pt for 15 epochs with batch size 32 and image size 224, then copies the best checkpoint to models/yolov5_sign_cls.pt (8.6 MB).
3. Evaluation: evaluate_model.py computes accuracy on the test data and generates the confusion matrix.
4. Inference engine: predictor.py loads the checkpoint and class_labels.json; preprocessing.py applies the PyTorch transforms.
5. Stabilisation: prediction_stabilizer.py implements the state machine with the thresholds described in Section 6.2.
6. Speech: text_to_speech.py runs a background worker thread so that speech is continuous and non-blocking, with a live “Dictating” indicator.
7. User interface: main_window.py coordinates the camera panel, conversation panel and library panel; the camera runs on its own thread and inference on a 200 ms timer.
8. Testing: unit and integration tests cover model loading, the stabiliser, preprocessing, the transcript manager, speech and the UI.

7.3 Project Structure

The code is organised into app/core (settings, conversation manager), app/vision (camera, predictor, stabiliser, preprocessing), app/speech (text-to-speech) and app/ui (main window, camera, conversation and library panels), together with scripts/ for dataset preparation, training and evaluation, models/ for the checkpoint and labels, and tests/.8. Results and Discussion

8.1 Quantitative Performance

Table 4: Model performance

Metric	Value
Validation Top-1 accuracy	100.0%
Validation Top-5 accuracy	100.0%
CPU inference latency	≈ 5.7 ms per image
GPU (CUDA) inference latency	≈ 0.7 ms per image
Model size	8.6 MB
Parameters / GFLOPs	4,206,399 / 10.4
Automated tests	113 passing
Test-set accuracy (279 images)	[fill from evaluate_model.py output]

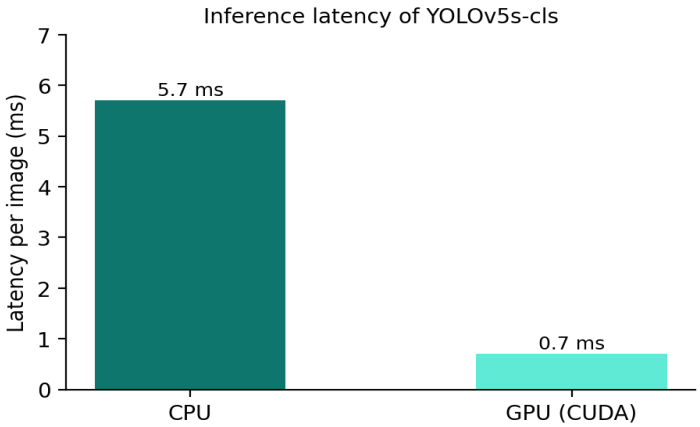
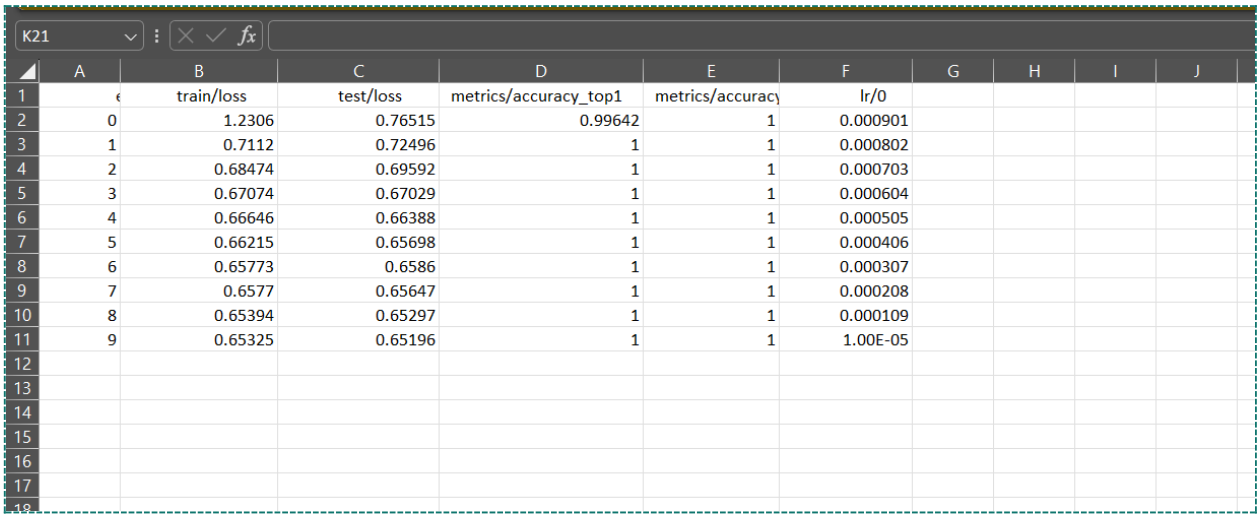


Figure 5: Inference latency on CPU and GPU

At about 5.7 ms per frame on a CPU, a single inference takes far less than the 200 ms interval between predictions, so the system leaves plenty of headroom for the camera thread, user interface and speech engine and responds smoothly in real time. A GPU lowers the latency further to about 0.7 ms but is not required.

8.2 Training Results and Confusion Matrix



	A	B	C	D	E	F	G	H	I	J
1		train/loss	test/loss	metrics/accuracy_top1	metrics/accuracy	lr/0				
2	0	1.2306	0.76515	0.99642	1	0.000901				
3	1	0.7112	0.72496	1	1	0.000802				
4	2	0.68474	0.69592	1	1	0.000703				
5	3	0.67074	0.67029	1	1	0.000604				
6	4	0.66646	0.66388	1	1	0.000505				
7	5	0.66215	0.65698	1	1	0.000406				
8	6	0.65773	0.6586	1	1	0.000307				
9	7	0.6577	0.65647	1	1	0.000208				
10	8	0.65394	0.65297	1	1	0.000109				
11	9	0.65325	0.65196	1	1	1.00E-05				
12										
13										
14										
15										
16										
17										

Figure 6: Training and validation curves

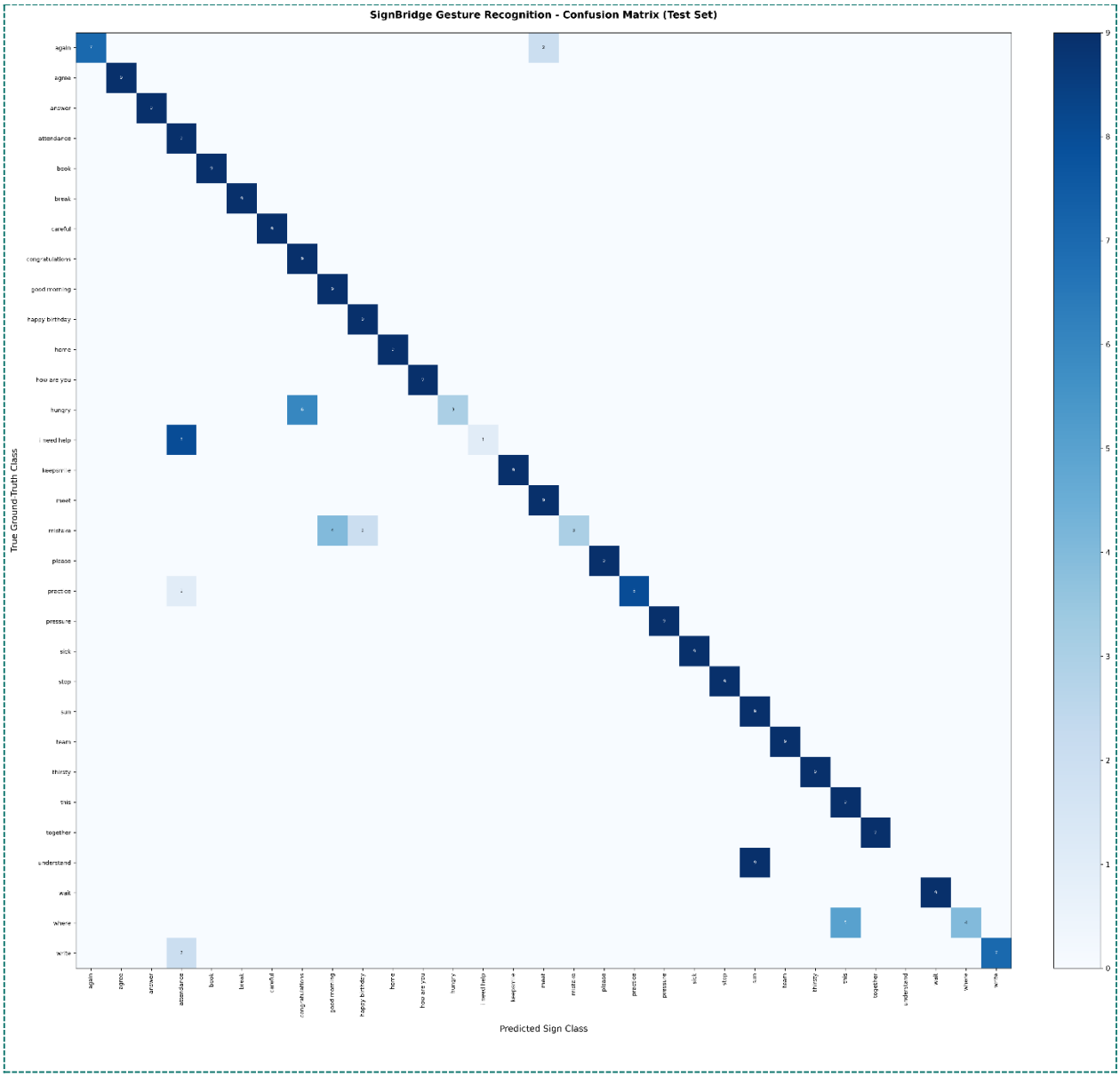


Figure 7: Confusion matrix of the 31 classes

The model reached perfect Top-1 and Top-5 accuracy on the validation split, meaning that every validation image was assigned to the correct class. The confusion matrix is therefore expected to be strongly diagonal, with all 31 signs recognised without confusion [verify against your matrix and comment on any off-diagonal entries if present].

8.3 Input vs. Output and Application Screenshots

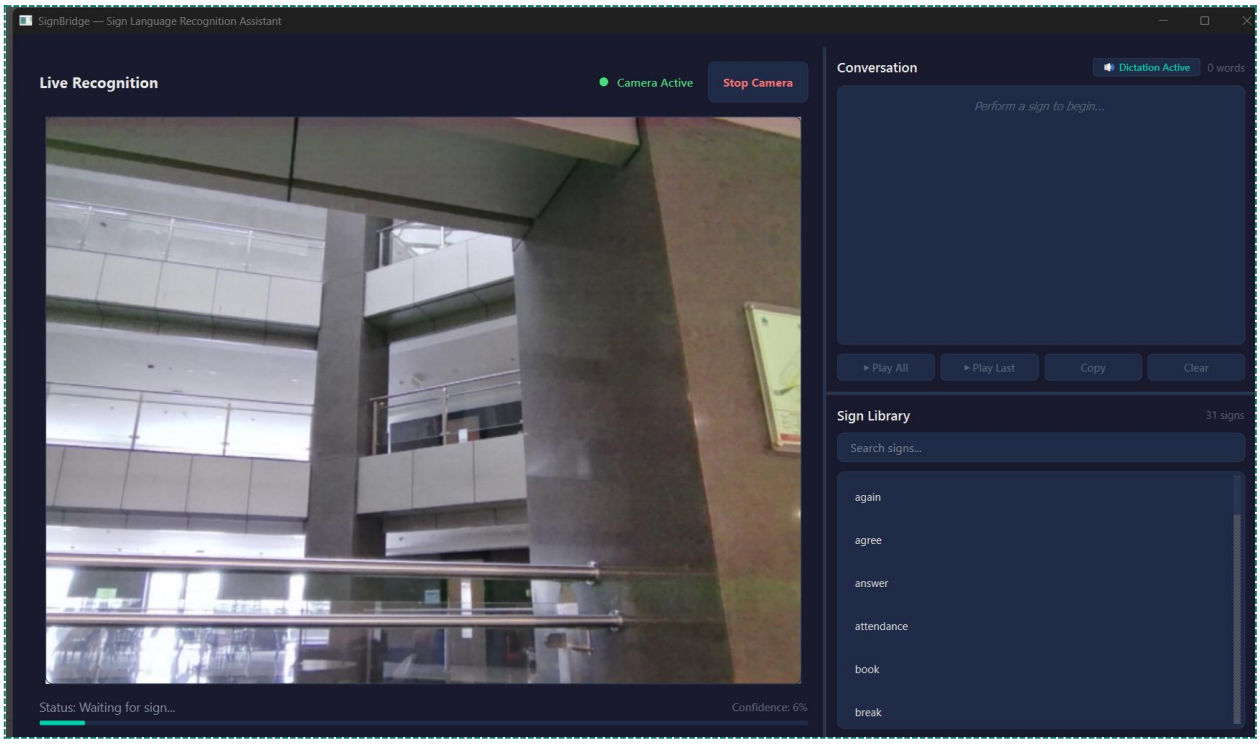


Figure 8: SignBridge main window with live recognition

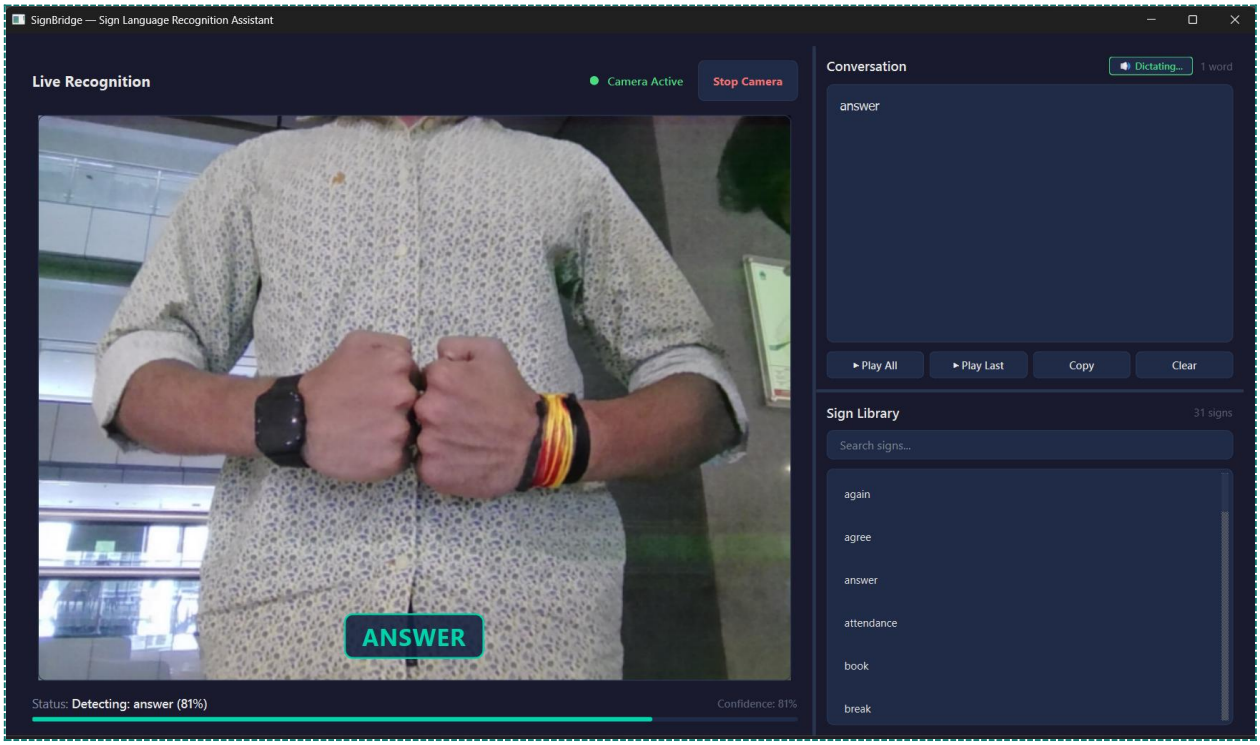


Figure 9: Input frames and corresponding recognised output

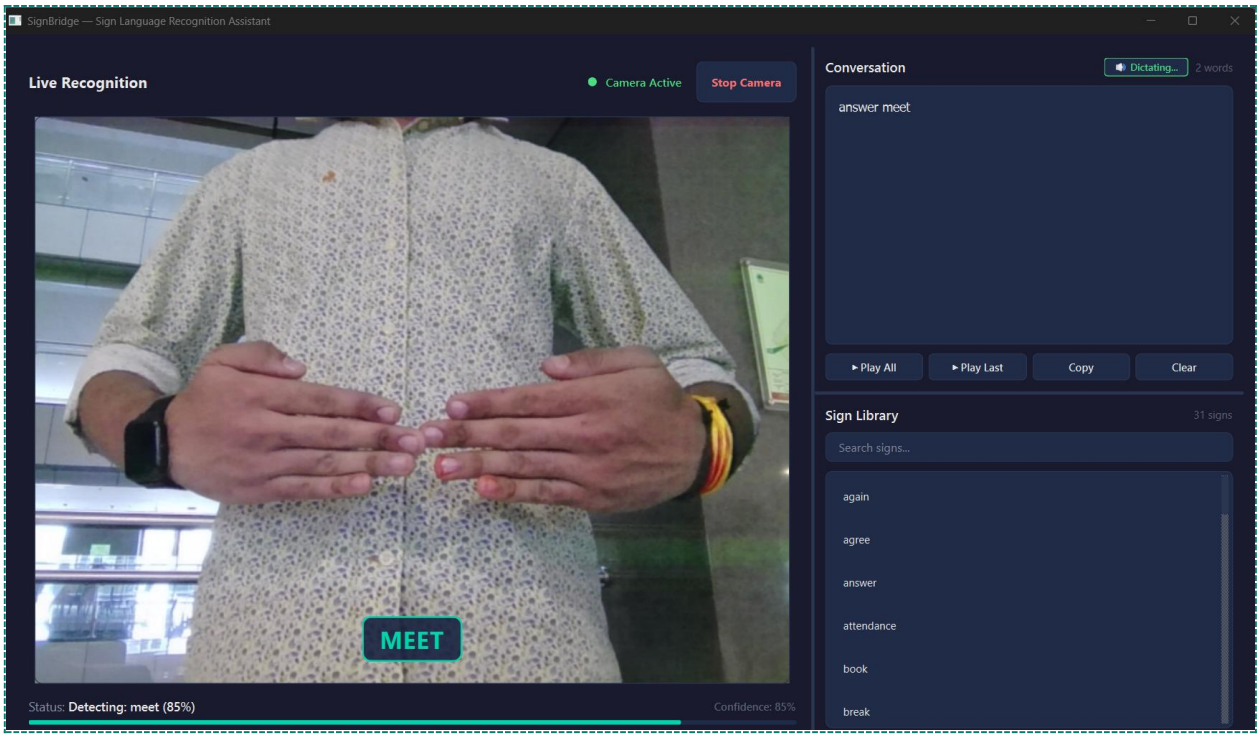


Figure 10: Conversation transcript and searchable sign library

8.4 Effect of Temporal Stabilisation

Without smoothing, a single mis-classified frame would immediately add a wrong word, and a sign held for one second would be added several times. The state machine removes both problems: a word must win at least 3 of 5 frames with an average confidence of at least 40%, and a short cooldown prevents the same sign from being re-triggered by accident. The design therefore trades a small delay of roughly one second for much more reliable output.

8.5 Discussion

- The transfer-learning approach with a lightweight YOLOv5s-cls model is sufficient for a 31-class sign vocabulary and runs comfortably on a CPU.
- Combining a frame-level classifier with a temporal stabiliser is what makes the output usable in practice, as the stabiliser handles noise that accuracy on still images does not reveal.
- Speech output through native engines gives immediate audible feedback, which supports two-way conversation with hearing users.

9. Challenges / Limitations

9.1 Problems Encountered

- Flicker and duplicates: raw per-frame predictions changed rapidly and repeated words, which required the temporal stabiliser and cooldown.
- Continuous speech: calling the speech engine directly blocked the interface and dropped utterances, which was solved with a background worker thread and platform-specific engines.
- Mirror effect: the unflipped webcam image felt unnatural to the signer, so mirror correction was added.

9.2 Dataset Limitations

- Only 60 images per class (1,860 in total) and a limited vocabulary of 31 signs.
- The images were collected by the project team, so variation in signers, backgrounds, lighting, camera distance and skin tones is likely limited [edit to match how you collected the data].
- The 100% validation accuracy should be read with caution: validation images come from the same collection session as the training images and may be very similar to them, so accuracy on new users and new environments is expected to be lower.

9.3 Difficult Cases and Failure Cases

- Signs with similar hand shapes, partial occlusion, motion blur, poor lighting or cluttered backgrounds may be mis-classified [add any failures observed during live testing].
- Each frame is classified independently, so signs that depend on motion over time are only captured through their key pose.

9.4 Computational Limitations

Inference is fast, but the application relies on a webcam and, for speech, on Windows SAPI5 or macOS say; Linux speech output is not supported. Prediction every 200 ms and the stabilisation window introduce a delay of about one second before a word is spoken.

10. Conclusion

This project implemented SignBridge, a real-time sign language recognition assistant that turns live webcam video into text and spoken words. A YOLOv5s-cls classifier was fine-tuned on a self-collected dataset of 31 conversational signs, and a video pipeline of frame capture, sampling, mirror correction, colour conversion, resizing and normalisation feeds the network. A temporal state-machine stabiliser makes the predictions reliable, and a PySide6 interface provides the live view, running transcript, text-to-speech playback and searchable sign library.

The system achieved 100% validation Top-1 and Top-5 accuracy with about 5.7 ms latency on a CPU, and the code is supported by 113 passing automated tests and reproducible training and evaluation scripts. Through this work we learned how image pre-processing, transfer learning and temporal filtering combine in a practical video application, and that real-time behaviour depends as much on stabilisation, threading and user-interface design as on model accuracy.

In future work the dataset can be extended with more signers, backgrounds and lighting conditions, the vocabulary can be enlarged to include finger-spelling, and hand detection or landmark tracking and sequence models can be added to recognise dynamic signs and full sentences.

References

[1] G. Jocher et al., "YOLOv5 by Ultralytics," 2020. [Online]. Available: https://github.com/ultralytics/yolov5

[2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779–788.

[3] C.-Y. Wang et al., "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," in Proc. IEEE/CVF CVPR Workshops, 2020.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in Proc. IEEE CVPR, 2009, pp. 248–255.

[5] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," IEEE Trans. Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345–1359, 2010.

[6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[7] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[8] The Qt Company, "Qt for Python (PySide6) Documentation." [Online]. Available: https://doc.qt.io/qtforpython-6/

[9] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.

[10] Sign_language_Generation_Using_YOLO_v5.ipynb — reference notebook adapted for the training methodology of this project.